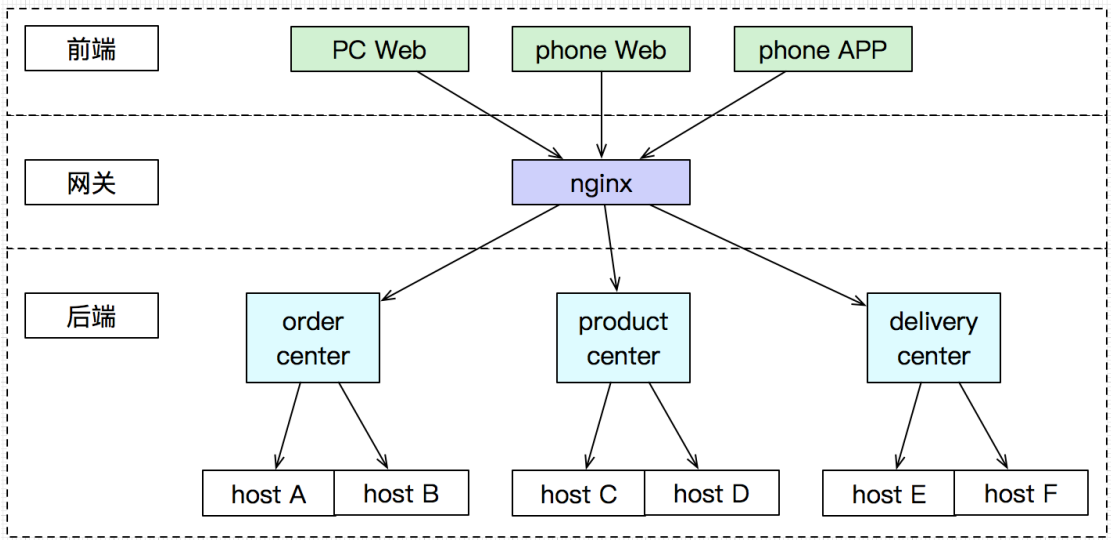


网关在架构中位于流量入口的位置，网关的重要性体现在负载均衡。
网关以常用的 nginx 为例。

网关架构图如下。



整体架构分为前端、网关、后端。
网关承上启下，核心在于区分后端多个应用。
区分后端应用的方法，包括使用二级域名区分、使用 path 前缀区分。

汇总每个应用的机器列表

1、静态法，手动指定机器 IP，把机器 IP 列表写到 nginx 的配置文件。

比如，订单中心包括 192.168.1.100、192.168.1.101。
nginx 配置示例为

```
upstream product_center {
    server 192.168.1.100:8080 weight=3; # 权重为 3
    server 192.168.1.101:8080;         # 默认权重为 1
}
```

2、动态法，机器启用应用后自动上报注册中心，nginx 查询注册中心，依赖 nginx-upsync-module 模块。

比如，商品中心使用 product_center 标识，包括 192.168.1.105、192.168.1.106。

nginx 配置示例为

```
upstream product_center {
    # 从 Zookeeper 获取服务列表
    upsync 127.0.0.1:2181/path/to/services/product_center upsync_timeout=6m upsync_interval=500ms
    upsync_type=zookeeper strong_dependency=off;

    # 将服务列表保存到本地文件
```

```
upsync_dump_path /usr/local/nginx/conf/servers/product_center_hosts.conf;
}
```

方案 1：使用二级域名区分后端应用

给每个后端应用分配 1 个二级域名：

后端应用	二级域名
订单中心	order.danceone.com
商品中心	product.danceone.com
物流中心	delivery.danceone.com

nginx 配置示例为：

```
server {
    listen 80;
    server_name order.danceone.com; # 二级域名

    location / {
        proxy_pass http://order_center; # 转发到 应用
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

server {
    listen 80;
    server_name product.danceone.com; # 二级域名

    location / {
        proxy_pass http://product_center; # 转发到 应用
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

server {
    listen 80;
    server_name delivery.danceone.com; # 二级域名

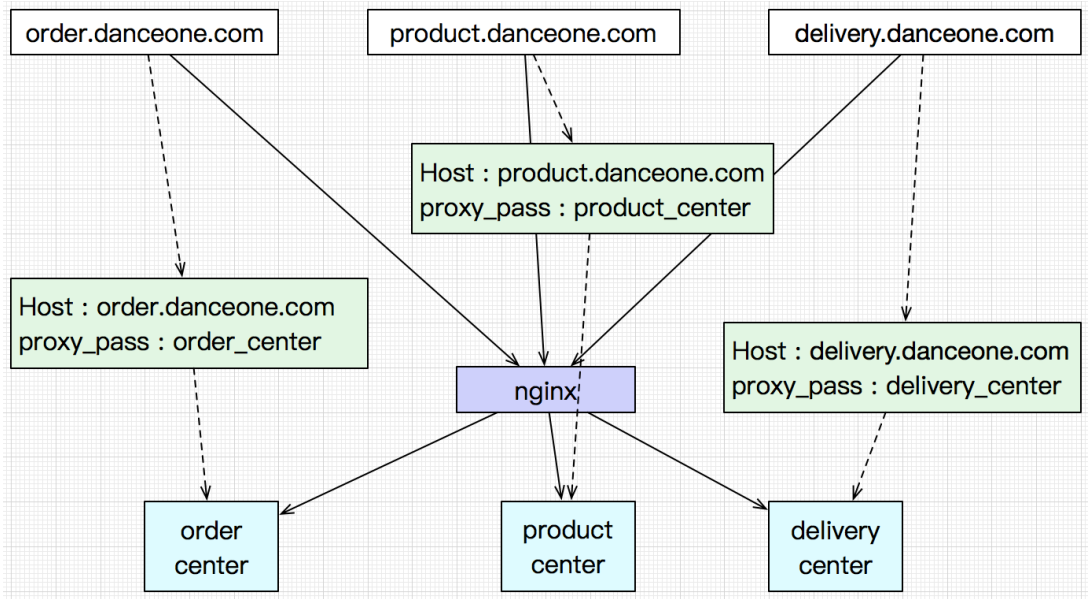
    location / {
        proxy_pass http://delivery_center; # 转发到 应用
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}
}
```

关键技术点：

- 使用 3 个 server 配置，分别对应 3 个后端应用。
- 每个 server 的逻辑：用 server_name 取到请求域名，用 proxy_pass 转发给后端应用。
- 使用 server_name 标识，取 http 请求 header Host。
- 发送 web 请求时，请求的域名会写入 header Host，网关可以判断此 header，知晓来源。
- 使用 proxy_pass 标识，转发到后端应用。后端应用使用前面步骤的 upstream 标识。

架构图：



方案 2：使用 path 前缀区分后端应用

多个后端应用共用 1 个二级域名，比如 www.danceone.com。

给每个后端应用分配 1 个 path 前缀：

后端应用	path 前缀
订单中心	/order
商品中心	/product
物流中心	/delivery

nginx 配置示例为：

```
# 配置 www.danceone.com 的 server 块
server {
    listen 80;
    server_name www.danceone.com; # 二级域名
```

```

# 订单中心的路由
location /order {
    proxy_pass http://order_center; # 转发到订单中心
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# 商品中心的路由
location /product {
    proxy_pass http://product_center; # 转发到商品中心
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

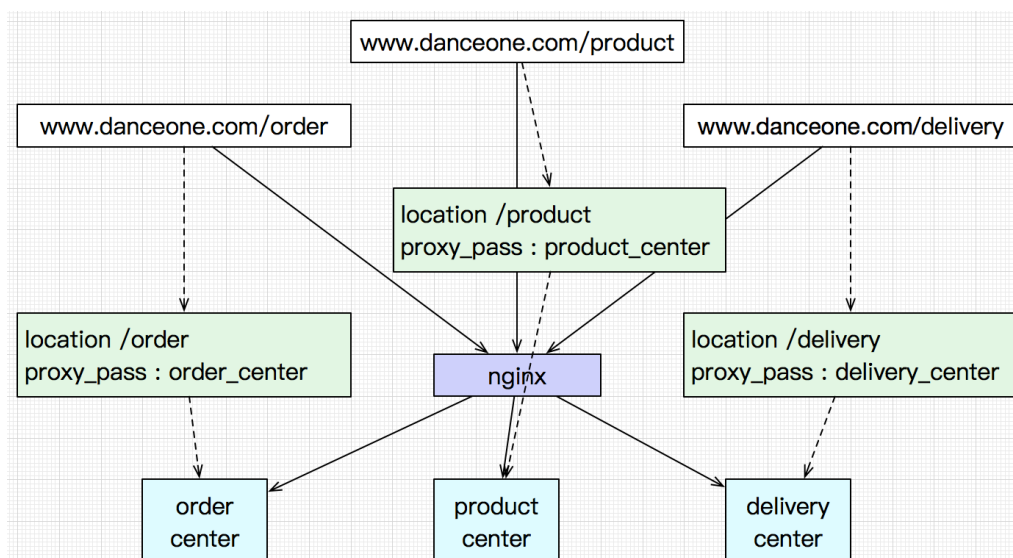
# 物流中心的路由
location /delivery {
    proxy_pass http://delivery_center; # 转发到物流中心
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

```

关键技术点：

- 因为共用 1 个二级域名，所以 server、server_name 只配 1 次，对应域名 www.danceone.com。
- 使用 location 区分 path 前缀，这里配置了 3 个 location，分别对应前缀 /order、/product、/delivery。
- 使用 proxy_pass 标识，转发到后端应用。1 个 location 对应 1 个 proxy_pass，共有 3 个 proxy_pass。

架构图：



nginx 的负载均衡源码

查看 nginx 的源码，找到负载均衡相关的代码文件。

nginx 提供多种负载均衡策略，默认使用轮询法。

- ngx_http_upstream_round_robin.c 轮询法
- ngx_http_upstream_hash_module.c 哈希法
- ngx_http_upstream_ip_hash_module.c IP 哈希法
- ngx_http_upstream_least_conn_module.c 最少连接
- ngx_http_upstream_random_module.c 随机法

轮询法的部分代码：

此处代码的含义为：遍历 peer 机器列表，排除不合法的 peer，使用 weight 权重找出一个合适的 peer。

```
static ngx_http_upstream_rr_peer_t *
ngx_http_upstream_get_peer(ngx_http_upstream_rr_peer_data_t *rrp)
{
    time_t          now;
    uintptr_t        m;
    ngx_int_t        total;
    ngx_uint_t        i, n, p;
    ngx_http_upstream_rr_peer_t  *peer, *best;

    now = ngx_time();

    best = NULL;
    total = 0;

#ifdef NGX_SUPPRESS_WARN
    p = 0;
#endif

    for (peer = rrp->peers->peer, i = 0;
         peer;
         peer = peer->next, i++)
    {
        n = i / (8 * sizeof(uintptr_t));
        m = (uintptr_t) 1 << i % (8 * sizeof(uintptr_t));

        if (rrp->tried[n] & m) {
            continue;
        }

        if (peer->down) {
            continue;
        }

        if (peer->max_fails
            && peer->fails >= peer->max_fails
            && now - peer->checked <= peer->fail_timeout)
```

```

    {
        continue;
    }

    if (peer->max_conns && peer->conns >= peer->max_conns) {
        continue;
    }

    peer->current_weight += peer->effective_weight;
    total += peer->effective_weight;

    if (peer->effective_weight < peer->weight) {
        peer->effective_weight++;
    }

    if (best == NULL || peer->current_weight > best->current_weight) {
        best = peer;
        p = i;
    }
}

if (best == NULL) {
    return NULL;
}

rrp->current = best;

n = p / (8 * sizeof(uintptr_t));
m = (uintptr_t) 1 << p % (8 * sizeof(uintptr_t));

rrp->tried[n] |= m;

best->current_weight -= total;

if (now - best->checked > best->fail_timeout) {
    best->checked = now;
}

return best;
}

```